

Universidad Autónoma de Santo Domingo

PRIMADA DE AMERICA / Fundada el 28 de octubre de 1538

Nombre:

Wilmy.

Apellidos:

Medina Batista.

Matrícula:

100681393.

Asignatura:

Laboratorio De Lenguaje De Programación III.

Tema:

Funcionalidades Del Spring Framework.

Sección:

01.

Profesor:

Radhames Silverio González.

Fecha:

1/11/2026.

ÍNDICE

INTRODUCCIÓN 3

DESARROLLO: 4

1. ¿Qué es Spring Framework? 4

2. Principales características y ventajas. 4

Principales características: 5

Ventajas de utilizar Spring: 5

3. Explicación de los módulos más importantes. 6

3.1. Spring Core: 6

3.2. Spring MVC: 7

3.3. Spring Boot: 7

3.4. Spring Data: 8

3.5. Spring Security: 8

4. Ejemplos de uso del framework en aplicaciones reales. 9

4.1. Plataforma de comercio electrónico: 9

4.2. Sistema de gestión universitaria: 9

4.3. Sistema de banca o servicios financieros: 10

4.4. Aplicaciones empresariales y microservicios: 10

4.5. Ejemplo ilustrativo de un controlador REST: 10

5. Conclusión personal sobre la importancia de Spring Framework en el desarrollo de software. 11

CONCLUSIÓN 13

REFERENCIAS BIBLIOGRÁFICAS 14

INTRODUCCIÓN

En el desarrollo de aplicaciones empresariales, una de las tareas más importantes consiste en construir sistemas que puedan organizarse, mantenerse y ampliarse sin que cada cambio represente un problema para el resto del programa. A medida que una aplicación aumenta sus funciones, también crecen sus necesidades de administrar datos, atender solicitudes, controlar el acceso de los usuarios y comunicarse con otros servicios. En Java, el Spring Framework ofrece una base de trabajo que ayuda a resolver estas necesidades mediante componentes y herramientas que pueden integrarse de acuerdo con los objetivos de cada proyecto.

Spring Framework es una plataforma amplia para el desarrollo de aplicaciones basadas en Java. Su propósito no es sustituir el lenguaje, sino proporcionar infraestructura para tareas comunes del software empresarial. Entre sus capacidades se encuentran la inyección de dependencias, la administración de transacciones, el acceso a datos, el desarrollo web, la programación orientada a aspectos y el soporte para pruebas. De esta manera, los programadores pueden concentrarse en las reglas y funciones particulares del negocio, sin tener que construir desde cero todos los mecanismos técnicos que necesita una aplicación (Spring, s. f.-a).

Una de las razones por las que Spring se utiliza en distintos tipos de proyectos es su diseño modular. Esto significa que no es obligatorio incorporar todas sus herramientas en una misma aplicación. Un equipo puede utilizar el contenedor central para administrar objetos, añadir Spring MVC para crear una aplicación web, integrar Spring Data para trabajar con una base de datos y emplear Spring Security para proteger las operaciones. Spring Boot, por su parte, facilita la configuración y puesta en marcha de aplicaciones construidas sobre el ecosistema Spring.

El propósito de esta investigación es describir qué es Spring Framework, explicar sus características y ventajas, y presentar el funcionamiento general de cinco de sus componentes o proyectos más conocidos: Spring Core, Spring MVC, Spring Boot, Spring Data y Spring Security. También se incluyen ejemplos de situaciones empresariales en las que estas herramientas pueden utilizarse, con el fin de comprender cómo contribuyen a desarrollar sistemas organizados, seguros y fáciles de mantener.

DESARROLLO:

1. ¿Qué es Spring Framework?

Spring Framework es un framework de código abierto para Java que proporciona un modelo de programación y una infraestructura completa para construir aplicaciones, especialmente aquellas que necesitan manejar procesos empresariales. Un framework es un conjunto de herramientas, bibliotecas y convenciones que ofrece una estructura de trabajo para resolver problemas frecuentes. En el caso de Spring, esa estructura permite administrar los componentes de una aplicación y conectarlos entre sí sin que el programador tenga que controlar manualmente cada detalle de su creación y funcionamiento.

Su elemento central es el contenedor de Inversión de Control (IoC), que administra los objetos de la aplicación y sus dependencias. En lugar de que una clase cree por su cuenta todos los objetos que necesita, Spring puede encargarse de proporcionar esas dependencias. Este mecanismo se conoce como Inyección de Dependencias (DI). El resultado es que las clases quedan menos atadas a implementaciones concretas y pueden probarse, modificarse o reutilizarse con mayor facilidad (Spring, s. f.-b).

Spring también ofrece soporte para otras tareas que suelen aparecer en sistemas empresariales, como el manejo de transacciones, el acceso a bases de datos, la validación, la integración con servicios externos, la creación de aplicaciones web y la seguridad. Estas funciones se encuentran organizadas en módulos y proyectos que pueden utilizarse juntos o por separado, según las necesidades del sistema.

Es importante distinguir Spring Framework de Spring Boot. Spring Framework es la base que aporta capacidades como el contenedor IoC, el acceso a datos y Spring MVC. Spring Boot es un proyecto del ecosistema Spring que simplifica la configuración y permite iniciar aplicaciones con menos trabajo manual. Por tanto, no son nombres para la misma herramienta: Spring Boot facilita el uso de Spring y de otras bibliotecas compatibles (Spring, s. f.-c).

2. Principales características y ventajas.

Spring reúne varias características que buscan mejorar la organización del código y simplificar las tareas habituales del desarrollo de software. No todas las aplicaciones necesitan utilizar cada una de ellas, pero el framework permite seleccionar las capacidades que aportan valor al proyecto.

Principales características:

- **Inversión de Control e Inyección de Dependencias:** El contenedor de Spring crea y administra los objetos de la aplicación, conocidos como beans, y proporciona a cada uno los componentes que necesita. Esto reduce la creación manual de dependencias y facilita sustituir una implementación por otra.
- **Programación modular:** Spring está organizado en módulos y proyectos. El desarrollador puede incorporar solo las partes necesarias, evitando añadir componentes que no tienen una función dentro del sistema.
- **Programación orientada a aspectos (AOP):** Permite separar funciones que afectan a diferentes partes de una aplicación, como el registro de actividad, la seguridad o determinadas operaciones de administración. Así se evita repetir la misma lógica en numerosas clases.
- **Administración de transacciones:** Ofrece mecanismos para controlar operaciones que deben ejecutarse como una unidad, especialmente cuando se realizan cambios en una base de datos. Esto ayuda a mantener la consistencia de la información.
- **Integración con tecnologías de datos:** Proporciona soporte para JDBC y para tecnologías de persistencia y mapeo objeto-relacional, además de integrarse con proyectos como Spring Data.
- **Desarrollo web:** Incluye Spring MVC para aplicaciones web basadas en el modelo de solicitudes y respuestas, y Spring WebFlux para aplicaciones que necesitan un enfoque reactivo.
- **Soporte para pruebas:** Incluye herramientas para probar componentes, contextos de Spring y aplicaciones web, lo que permite comprobar el comportamiento del sistema de forma más organizada.
- **Configuración flexible:** Permite definir componentes mediante anotaciones, clases de configuración y otros mecanismos, sin depender obligatoriamente de archivos XML.

Ventajas de utilizar Spring:

- **Mejor organización del proyecto:** Al separar responsabilidades y administrar las dependencias, el código puede distribuirse en componentes con funciones más claras.
- **Facilidad de mantenimiento:** Una modificación en una parte del sistema puede realizarse con menor impacto en otras áreas cuando las dependencias están bien diseñadas.
- **Reutilización de componentes:** Los servicios, repositorios y otros objetos pueden utilizarse desde diferentes partes de la aplicación.
- **Pruebas más sencillas:** La posibilidad de sustituir dependencias facilita crear pruebas unitarias y comprobar componentes sin tener que ejecutar siempre el sistema completo.
- **Productividad:** El framework proporciona soluciones para tareas comunes, por lo que el equipo puede dedicar más tiempo a las necesidades específicas del negocio.

- **Integración con el ecosistema Java:** Spring puede trabajar con bases de datos, servidores web, sistemas de mensajería y otras bibliotecas utilizadas en proyectos empresariales.
- **Escalabilidad y flexibilidad:** Su estructura permite comenzar con una aplicación sencilla y ampliar sus capacidades a medida que aparecen nuevos requisitos, siempre que la arquitectura se diseñe correctamente.

Sin embargo, Spring no garantiza por sí solo que una aplicación sea segura, rápida o fácil de mantener. Los resultados dependen de las decisiones de diseño, de la calidad del código y de la configuración realizada por el equipo. También existe una curva de aprendizaje, porque es necesario comprender el contenedor, las anotaciones, la configuración y la relación entre los diferentes proyectos del ecosistema.

3. Explicación de los módulos más importantes.

El ecosistema Spring contiene numerosos módulos y proyectos. En esta investigación se explican cinco de los más conocidos por su utilidad en aplicaciones empresariales. Conviene aclarar que Spring Core y Spring MVC forman parte del Spring Framework, mientras que Spring Boot, Spring Data y Spring Security son proyectos relacionados que se integran con el framework y amplían sus capacidades.

3.1. Spring Core:

Spring Core es la parte fundamental sobre la que se apoyan muchas de las demás funciones de Spring. Su responsabilidad principal es ofrecer el contenedor de Inversión de Control, que administra los objetos de la aplicación y controla cómo se crean, configuran y relacionan. Los objetos administrados por el contenedor reciben el nombre de beans.

Por ejemplo, en un sistema de ventas puede existir un servicio encargado de procesar pedidos y otro componente responsable de consultar los productos. El servicio de pedidos necesita utilizar el componente de productos. En lugar de crear directamente una instancia de esa clase dentro del servicio, Spring puede inyectarla mediante el constructor. Esto disminuye el acoplamiento y permite cambiar o simular la dependencia durante las pruebas.

La Inyección de Dependencias puede realizarse de distintas maneras, aunque la inyección por constructor suele ser una opción clara cuando una clase necesita dependencias obligatorias. El contenedor también administra el ciclo de vida de los beans y permite configurar sus relaciones. Además, las tecnologías centrales incluyen eventos, recursos, validación, conversión de tipos y soporte para programación orientada a aspectos (Spring, s. f.-b).

En una aplicación empresarial, Spring Core ayuda a organizar los servicios y componentes que contienen las reglas del negocio. Por ejemplo, una plataforma de inventario puede tener servicios para registrar productos, actualizar existencias y calcular disponibilidad. El

contenedor se encarga de conectar esos servicios con los repositorios que consultan la base de datos.

3.2. Spring MVC:

Spring MVC es el módulo web del Spring Framework que permite construir aplicaciones web siguiendo el patrón Modelo-Vista-Controlador. Se basa en la API Servlet y utiliza un componente central llamado DispatcherServlet, que recibe las solicitudes HTTP y coordina su procesamiento mediante otros componentes configurables (Spring, s. f.-d).

En términos generales, cuando un usuario visita una dirección o envía un formulario, la solicitud llega al DispatcherServlet. Este identifica el controlador que debe atenderla. El controlador ejecuta la operación necesaria, puede consultar servicios o datos y luego devuelve una vista o una respuesta directa. En una aplicación tradicional puede generar una página HTML; en una API REST puede devolver información en formato JSON.

Los controladores suelen declararse mediante anotaciones como @Controller o @RestController. Las rutas se asocian a métodos mediante anotaciones como @GetMapping y @PostMapping. Esto permite definir qué operación se ejecuta cuando se consulta información o se envían datos al servidor.

Un ejemplo de uso sería el portal web de una universidad. Spring MVC puede recibir una solicitud para consultar las asignaturas de un estudiante, llamar al servicio correspondiente y devolver una página con los resultados. También puede recibir los datos de un formulario de inscripción y coordinar su validación y registro.

3.3. Spring Boot:

Spring Boot es un proyecto del ecosistema Spring diseñado para facilitar la creación de aplicaciones independientes y listas para ejecutarse. Su objetivo es reducir la configuración inicial que normalmente requiere un proyecto Spring. Para lograrlo, ofrece autoconfiguración, dependencias agrupadas llamadas starters y servidores web integrados, entre otras funciones (Spring, s. f.-c).

La autoconfiguración permite que Spring Boot configure determinados componentes de acuerdo con las bibliotecas disponibles y las propiedades del proyecto. Los starters agrupan dependencias habituales para una función concreta. Por ejemplo, al utilizar un starter web se incorporan componentes necesarios para crear una aplicación web, sin tener que buscar y añadir manualmente cada biblioteca relacionada.

Otra característica es que una aplicación puede ejecutarse como un programa independiente, normalmente desde un método principal de Java, y puede incluir un servidor integrado como Tomcat. Además, Spring Boot ofrece funciones de producción mediante Spring Boot Actuator, que permiten consultar información de salud, métricas y otros datos de administración cuando se configuran adecuadamente.

En una empresa, Spring Boot resulta útil para crear rápidamente una API de pedidos, un servicio de inventario o una aplicación interna de recursos humanos. También se utiliza con frecuencia como base para construir microservicios, aunque no obliga a que toda aplicación creada con él tenga esa arquitectura.

3.4. Spring Data:

Spring Data es un conjunto de proyectos que busca simplificar el acceso a diferentes tipos de almacenes de datos. Proporciona una forma consistente de trabajar con datos relacionales y no relacionales, y reduce parte del código repetitivo que normalmente se necesita para consultar, guardar, actualizar y eliminar registros (Spring, s. f.-e).

Una de sus características conocidas es el uso de repositorios. Un repositorio representa una interfaz mediante la cual una aplicación puede realizar operaciones sobre una entidad o conjunto de datos. En determinados módulos, Spring Data puede generar implementaciones de operaciones comunes a partir de la interfaz y de los métodos que declara el desarrollador.

Por ejemplo, en una tienda virtual se puede definir un repositorio de productos con operaciones para buscar todos los productos, localizar uno por su identificador o consultar los que pertenecen a una categoría. Con Spring Data JPA, esas operaciones pueden trabajar con una base de datos relacional mediante JPA. También existen proyectos específicos para tecnologías como MongoDB, Redis, Cassandra y otras.

Spring Data no reemplaza a las bases de datos ni significa que todas ellas funcionen de la misma manera. Cada módulo se adapta al tipo de almacenamiento correspondiente. Su ventaja es ofrecer abstracciones y convenciones que hacen más uniforme el acceso a los datos dentro de una aplicación Spring.

3.5. Spring Security:

Spring Security es el proyecto que proporciona mecanismos de seguridad para aplicaciones Spring. Sus capacidades principales incluyen autenticación y autorización. La autenticación permite comprobar quién es el usuario que intenta acceder; la autorización determina qué recursos u operaciones puede utilizar ese usuario una vez que se ha identificado (Spring, s. f.-f; Spring, s. f.-g).

Por ejemplo, en un sistema bancario, un cliente debe iniciar sesión para consultar sus cuentas. Sin embargo, no todos los usuarios pueden realizar las mismas operaciones. Un cliente puede consultar sus propios movimientos, mientras que un empleado autorizado podría tener acceso a funciones administrativas. Spring Security permite establecer reglas para proteger rutas, métodos y recursos según los permisos definidos.

También ofrece integración con mecanismos de autenticación, manejo de contraseñas, protección de solicitudes y soporte para estándares y protocolos de seguridad. Su

configuración debe realizarse con cuidado: una aplicación puede utilizar Spring Security y aun así quedar expuesta si establece permisos demasiado amplios, almacena credenciales de forma incorrecta o no valida adecuadamente las operaciones.

En proyectos empresariales, este componente es importante porque permite incorporar controles de acceso como parte de la arquitectura de la aplicación, en lugar de desarrollar desde cero todos los mecanismos de autenticación y autorización.

4. Ejemplos de uso del framework en aplicaciones reales.

Spring puede utilizarse en distintos sectores porque sus herramientas cubren necesidades habituales de los sistemas empresariales. Los siguientes ejemplos representan escenarios concretos de aplicación; muestran cómo podrían combinarse los módulos, sin afirmar que una empresa determinada utilice exactamente esta arquitectura.

4.1. Plataforma de comercio electrónico:

Una tienda virtual necesita administrar clientes, productos, categorías, inventario, carritos y pedidos. Spring Boot puede servir como base para iniciar la aplicación y organizar su configuración. Spring MVC puede recibir solicitudes para consultar productos, registrar compras o revisar el estado de un pedido. Spring Data puede facilitar la comunicación con la base de datos donde se almacenan los productos y las órdenes. Spring Security puede proteger las cuentas de los clientes y restringir las funciones administrativas.

Cuando un cliente confirma una compra, el sistema puede validar los datos, consultar la disponibilidad del producto, registrar el pedido y actualizar el inventario. La administración de transacciones de Spring ayuda a coordinar las operaciones que deben mantenerse consistentes, de manera que un fallo no deje la información en un estado incorrecto.

4.2. Sistema de gestión universitaria:

Una institución educativa puede desarrollar un sistema para gestionar estudiantes, docentes, asignaturas, calificaciones e inscripciones. Spring Core permite organizar los servicios que contienen las reglas del sistema, como comprobar si un estudiante cumple los requisitos para inscribirse. Spring MVC puede atender las solicitudes de los portales web y Spring Data puede consultar los registros almacenados.

Spring Security permite diferenciar los permisos de estudiantes, profesores y administradores. Un estudiante podría consultar sus calificaciones, un profesor registrar notas de las asignaturas que imparte y un administrador gestionar las matrículas. Así, el sistema controla tanto la información disponible como las acciones que cada tipo de usuario puede realizar.

4.3. Sistema de banca o servicios financieros:

Una aplicación financiera necesita controlar el acceso a información privada y registrar operaciones sensibles. Spring Security puede encargarse de la autenticación y de aplicar reglas de autorización. Spring Data puede facilitar el acceso a la información de cuentas y movimientos, mientras que la administración de transacciones contribuye a mantener la consistencia de las operaciones.

En este tipo de sistema, Spring es una parte de la solución técnica, no una garantía automática de seguridad. También se necesitan controles adicionales, auditorías, cifrado, supervisión, validaciones y políticas de protección de datos. La configuración debe revisarse cuidadosamente debido a la importancia de la información manejada.

4.4. Aplicaciones empresariales y microservicios:

Una organización puede dividir una plataforma grande en servicios independientes, por ejemplo, un servicio de clientes, otro de facturación y otro de inventario. Spring Boot facilita crear y ejecutar cada servicio con una configuración más sencilla. Spring Framework aporta capacidades de inyección de dependencias, acceso a datos y comunicación web, mientras que otros proyectos del ecosistema pueden ayudar con mensajería, configuración distribuida o comunicación entre servicios.

Esta organización puede facilitar el despliegue y la evolución de partes del sistema de manera independiente. Sin embargo, los microservicios también introducen complejidad de comunicación, monitoreo y administración, por lo que no siempre son la mejor opción para aplicaciones pequeñas.

4.5. Ejemplo ilustrativo de un controlador REST:

El siguiente fragmento muestra una forma sencilla de utilizar Spring MVC en una aplicación Spring Boot. El controlador recibe una solicitud HTTP para consultar un producto y devuelve una respuesta. Se presenta con fines educativos y supone que existe un servicio ProductoService que contiene la operación de búsqueda.

```
@RestController
@RequestMapping("/api/productos")
public class ProductoController {
private final ProductoService productoService;

public ProductoController(ProductoService productoService) {
this.productoService = productoService;

}

@GetMapping("/{id}")
```

```
public Producto buscarPorId(@PathVariable Long id) {
return productoService.buscarPorId(id);

}

}
```

En este ejemplo, @RestController indica que la clase atiende solicitudes web y que sus resultados se devuelven como datos de respuesta. @RequestMapping establece la ruta base, mientras que @GetMapping define la operación para consultar un producto por su identificador. El constructor recibe ProductoService mediante inyección de dependencias, una capacidad proporcionada por el contenedor de Spring. La clase delega la búsqueda al servicio, en lugar de concentrar en el controlador toda la lógica del negocio.

En una aplicación completa también habría que definir la clase Producto, el servicio, el repositorio, el tratamiento de errores y las reglas de validación. El ejemplo permite observar cómo colaboran Spring Core, mediante la inyección de dependencias, y Spring MVC, mediante el manejo de solicitudes HTTP.

5. Conclusión personal sobre la importancia de Spring Framework en el desarrollo de software.

Después de estudiar Spring Framework, considero que es una herramienta de gran importancia para quienes se están formando en el desarrollo de aplicaciones con Java, porque permite comprender cómo se construyen sistemas que necesitan organizar diferentes funciones y trabajar con información de manera constante. Antes de conocer sus componentes, puede parecer que desarrollar una aplicación empresarial consiste solamente en escribir clases y métodos; sin embargo, al estudiar Spring se puede observar que también es necesario administrar dependencias, controlar transacciones, atender solicitudes, acceder a bases de datos y proteger los recursos.

Uno de los aspectos que más considero valiosos es la Inyección de Dependencias de Spring Core, porque ayuda a que las clases no tengan que crear por sí mismas todos los objetos que necesitan. Esto permite mantener una mejor separación de responsabilidades y facilita realizar cambios o pruebas. También me parece importante Spring Boot, porque reduce parte del trabajo de configuración y permite comenzar un proyecto de una forma más organizada.

Comprender la función de Spring MVC, Spring Data y Spring Security también permite reconocer que una aplicación empresarial necesita más que una interfaz. Debe recibir solicitudes, procesar reglas, guardar información y establecer quién puede acceder a cada función. Cada uno de estos proyectos contribuye a resolver una parte del problema y, al trabajar juntos, pueden formar una solución más completa.

En lo personal, entiendo que aprender Spring Framework no significa memorizar todas sus anotaciones o configuraciones, sino comprender para qué sirve cada herramienta y en qué momento conviene utilizarla. Este conocimiento puede ayudarme a desarrollar proyectos más estructurados y a prepararme para trabajar con tecnologías utilizadas en entornos profesionales. Por eso considero que Spring es una base valiosa dentro del aprendizaje de Java y del desarrollo de software empresarial.

CONCLUSIÓN

Spring Framework es una plataforma fundamental dentro del ecosistema Java porque proporciona una infraestructura que facilita la construcción de aplicaciones empresariales. Su principal aporte consiste en ofrecer mecanismos para organizar los componentes de un sistema, administrar sus dependencias y resolver tareas técnicas que aparecen con frecuencia durante el desarrollo. Gracias a esta estructura, los equipos pueden concentrarse en implementar las necesidades particulares de cada organización.

A través de sus características, Spring permite trabajar con Inversión de Control, Inyección de Dependencias, programación orientada a aspectos, administración de transacciones, acceso a datos, desarrollo web e integración con herramientas de prueba. Estas capacidades contribuyen a crear aplicaciones cuyo código puede mantenerse y ampliarse de manera más ordenada, siempre que se apliquen buenas prácticas de diseño y configuración.

Los módulos y proyectos estudiados cumplen funciones complementarias. Spring Core administra los objetos y sus relaciones; Spring MVC permite atender solicitudes web y construir aplicaciones siguiendo el patrón Modelo-Vista-Controlador; Spring Boot simplifica la configuración y ejecución de proyectos; Spring Data facilita el acceso a diferentes sistemas de almacenamiento; y Spring Security incorpora mecanismos para autenticar usuarios y controlar sus permisos. La combinación de estas herramientas permite desarrollar soluciones para comercios electrónicos, instituciones educativas, entidades financieras y sistemas empresariales.

En definitiva, Spring Framework continúa siendo relevante porque responde a necesidades que se presentan en proyectos de software de diferentes tamaños: organizar el código, conectar componentes, administrar información y proteger las operaciones. No sustituye la planificación ni las decisiones del desarrollador, pero ofrece una base tecnológica que ayuda a construir aplicaciones Java más estructuradas, flexibles y preparadas para evolucionar. Conocer sus funcionalidades permite comprender mejor el desarrollo empresarial moderno y contar con herramientas útiles para enfrentar proyectos reales.

REFERENCIAS BIBLIOGRÁFICAS

Spring. (s. f.-a). Spring Framework. Spring Projects. https://spring.io/projects/spring-framework/

Spring. (s. f.-b). Core technologies. Spring Framework Reference Documentation. https://docs.spring.io/spring/reference/core.html

Spring. (s. f.-c). Spring Boot. Spring Projects. https://spring.io/projects/spring-boot/

Spring. (s. f.-d). Spring Web MVC. Spring Framework Reference Documentation. https://docs.spring.io/spring/reference/web/webmvc.html

Spring. (s. f.-e). Spring Data. Spring Projects. https://spring.io/projects/spring-data/

Spring. (s. f.-f). Authentication. Spring Security Reference Documentation. https://docs.spring.io/spring-security/reference/features/authentication/

Spring. (s. f.-g). Authorization. Spring Security Reference Documentation. https://docs.spring.io/spring-security/reference/servlet/authorization/